There are five major strands of existing solutions for Kolam: formal grammars (array/graph/cycle), tile-based construction, L-systems, graph/Eulerian formulations, and AI/ML classification, plus several open-source generators and datasets that operationalize these ideas with Python, JavaScript (p5.js), MATLAB, and modern CNNs with attention modules for heritage documentation and recognition. Together, these show that both the "design grammar" and automated generation/classification of Kolams are well-studied, with concrete technology stacks and reproducible methods that can be adapted for SIH-ready solutions emphasizing cultural preservation and computational rigor.

Formal grammars

- Array grammars: Siromoney's classic "Array Grammars and Kolam" formalized Kolam generation using 2D array grammars with finite primitives and production rules, establishing a foundational grammar-based approach to synthesis of motifs on dot grids.
- Cycle/graph grammars: Follow-on work extended these ideas to cycle grammars for Kolam (Kahbi Kolam), connecting picture grammars to graph-based generative schemes for cyclic traces around dots and curve elements.
- Hexagonal arrays: Extensions to hexagonal array grammars capture non-rectilinear Kolam families, broadening the grammar formalism to hexagonal tessellations and associated pattern rules.
- Empirical grammar insights: Siromoney's broader program studied how practitioners parse Kolams in stages, supporting the notion of constituent units and generative "design grammar" learned and reproduced incrementally.

Tile-based construction

- 16-tile system: Raghavachary introduced a set of 16 decorated tiles assembled corner-to-corner to algorithmically produce continuous-loop Kolams, offering a modular, exploratory alternative to direct curve drawing on a dot grid.
- Motivation: The tile approach contrasts with prior characterizations via picture grammars and L-systems, aiming to simplify memorization and variation by composing from a finite tile lexicon that respects connectivity and loop closure.

L-systems for generation

- L-systems + turtle graphics: Kolam curves can be generated by parallel rewriting systems producing turtle commands, aligning with established uses of L-systems for fractal and self-similar forms and enabling iterative complexity control.
- Python Turtle implementations: Public exemplars show concrete axiom/rule sets (e.g., axiom FBFBFBFB with A→AFBFA, B→AFBFBFBFA at 45°) and drawing semantics for F, A, B to produce single-knot Kolams via Python Turtle Graphics.
- Single-knot/Eulerian viewpoint blended with L-systems: Detailed notes formalize Sikku/Kambi Kolam as a single uninterrupted line and relate the generated trace to Eulerian graph properties while providing L-system procedures and drawing rules, again in Python Turtle.

Graph/Eulerian formulations

- Eulerian model: Treat crossings as vertices and segments as edges; single-kambi Kolams correspond to Eulerian graphs where a single circuit traverses every edge exactly once, providing a clean graph-theoretic lens and algorithmic hooks for generation and verification.
- Algorithmic implication: With an Eulerian structure, standard linear-time Euler tour algorithms (e.g., Hierholzer) can be used to construct or validate single-stroke traversals over the induced graph from dot layouts and curve-connection rules.

Sequential encoding and lexicons

- Expanded gesture lexicon: Waring formalizes a sequential "language" for square loop Kolams (SLK), expanding the standard orthogonal gesture set to a 30-gesture lexicon including orthogonal, diagonal, transitional, and stylistic moves, greatly increasing coverage of real patterns and enabling digital encoding and regeneration from symbolic sequences.

AI/ML for classification and heritage documentation

- KolamNet: An attention-augmented EfficientNet model classifies Kolam motifs into multiple categories, demonstrating the viability of modern CNNs with attention for organized motif recognition on curated datasets.
- KolamNetV2: An EfficientNet backbone enhanced with Dense Channel Attention and Parallel Spatial-Channel modules achieves strong performance across 13 Kolam categories, explicitly framing the work as digital heritage documentation and emphasizing lightweight deployment.
- Datasets: Public datasets exist for Kolam classification, including curated sets on Roboflow and a Kaggle "one-stroke dotted (Pulli) Kolam" collection, enabling supervised learning, benchmarking, and potential generative modeling via time-series or image representations.

Open-source generators and tools

- p5.js/JavaScript generators: Community projects implement generative Kolams in p5.js, including parameterized sketches and plotter-friendly SVG export variants that adapt algorithmic Kolam code for CNC/pen plotting pipelines.
- Web-based random generator: A Random Kolam Generator site allows generation of pulli Kolams with control over size, symmetry, and intricacy, demonstrating a simple UI over algorithmic rule assembly in a browser context.
- MATLAB toolbox: A Kolam Generator suite provides propose_kolam1D and propose_kolam2D to produce code matrices and renderings, illustrating a matrix-encoded grammar and programmatic drawing workflow in MATLAB.
- Additional repos: Other repositories experiment with Kolam generation (e.g., Kolamora), alongside design resources and logo concepts inspired by Sikku Kolam patterns within Python and vector tools ecosystems.

Technology stacks observed

- Python Turtle Graphics: Used to render L-system–driven Kolams with explicit axiom/rule sets and turtle semantics for arcs and line segments, emphasizing iterative complexity and single-knot generation.

- JavaScript (p5.js): Used for browser-native procedural Kolam sketches and SVG export for pen plotters/CNC, enabling interactive controls and web deployment without plugins.
- MATLAB: Provided functions encode Kolams as matrices and draw them, suitable for rapid prototyping of grammar-based and symmetric pattern generation.
- Deep learning frameworks: EfficientNet backbones with channel/spatial attention modules for classification, designed to be efficient and performant for heritage documentation use cases.

Conceptual structures behind solutions

- Grammar-centric: Array/graph/cycle grammars specify terminals/primitives and production rules mapping dot grids to curve compositions, supporting formal proofs of generatability and systematic construction.
- Modular tiling: A minimal set of decorated tiles compose larger closed-loop patterns while enforcing local connectivity constraints, enabling combinatorial exploration and user-friendly construction.
- Rewriting systems: L-systems produce command strings that naturally map to turtle drawings, making self-similar, recursive Kolam variants easy to iterate and scale.
- Graph traversal: Interpreting Kolams as Eulerian circuits aligns generation with classical traversal algorithms and provides robust validation that a pattern is single-stroke and edge-complete.
- Symbolic sequencing: Gesture lexicons encode the artist's stroke repertoire into a sequential language that can be parsed, stored, and regenerated, bridging human practice and computational representation.
- Vision pipelines: Attention-augmented CNNs learn dot, line, curve, and spatial relationships to classify Kolam categories, advancing digital archiving and potential downstream tasks like retrieval and style transfer.

Strengths, limitations, and fit for SIH

- Grammars: Strong formal interpretability and provable coverage within defined families, but may require multiple grammar variants to capture diverse regional/non-orthogonal styles comprehensively.
- Tiles: Highly usable and modular for education and design variation, though tile sets must be carefully designed to preserve continuous loops and may constrain stylistic nuances compared to freehand curves.
- L-systems: Excellent for recursive/self-similar families with simple parameter control, but deriving rules from arbitrary exemplars is non-trivial without human-designed productions or learning-based inference.
- Eulerian graphs: Clean single-stroke guarantees and efficient algorithms, but mapping aesthetic curve rules to graph constraints requires careful modeling of crossings and local curve topology.

- Lexicon encoding: Human-aligned and extensible, yet requires comprehensive gesture sets and annotation to achieve broad coverage and reliable generation from sequences.
- ML classification: Powerful for documentation and categorization, but primarily recognizes rather than explicates generative rules, unless paired with structure discovery or program induction approaches.

What this means for SIH

- A strong solution can combine a grammar/tile/L-system generator with an Eulerian validator to ensure single-stroke correctness while exposing parameters for grid, symmetry, and style, leveraging Python Turtle or browser p5.js for fast visualization and interactivity.
- To meet "design grammar extraction," consider Waring's lexicon or Siromoney's grammar families to formalize rules discovered from exemplars, and optionally add a CNN classifier like KolamNetV2 to tag style/category for educational value and digital preservation.
- Available datasets and open-source generators can seed benchmarking and UI prototypes quickly, with MATLAB/p5.js/Python options aligning to team skills and demo needs for generative variety and explainable rule control.

If desired, a follow-up can map these threads into a concrete architecture: rule inference pipeline (lexicon/grammar), generator backend (L-system/tile/graph), Eulerian validation, and a lightweight classifier front-end for documentation, each grounded in the cited works and stacks above

Comparison snapshot

Approach	Representative work	Core idea	Typical stack	Inputs/Outputs	Where it shines	Caveats
Array/graph grammars	Siromoney et al. (Array/Cycle/Hexagonal) [5][10][11]	2D grammars and cycle/graph rules generate Kolams from primitives on grids [5][10]	Theoretical formalisms; can be implemented in any language [5]	Dot arrays → generated curve patterns [5]	Formal clarity, provable generability [5]	Multiple grammars needed for diverse styles [11]
Tile-based	Raghavachary (16 tiles) [3][13]	Compose closed-loop Kolams by assembling decorated tiles with consistent connections [3]	Any rendering stack; concept published via SIGGRAPH sketch [3]	Tile sequences → continuous loop drawings [3]	Usability and rapid variation [3]	Tile design biases and style constraints [3]

L-systems	Imaginary/Brunda, Python-Turtle [1][18][17]	Rewriting rules produce turtle commands to draw single-knot Kolams with arc/line semantics [1]	Python + Turtle graphics [1]	Axiom + productions → rendered Kolams [1]	Recursive complexity, easy parameterization [1]	Hard to infer rules from arbitrary exemplars [15]
Eulerian graph	Graph/Eulerian Kolam framing [4][19]	Model single-kambi Kolams as Euler tours over induced graphs and construct with tour algorithms [4][21]	Any graph library; linear-time Euler tour methods [21]	Dot layout → graph → tour/trace [4]	Single-stroke guarantees, algorithmic rigor [4]	Requires careful graph construction from curves [4]
Sequential lexicon	Waring (30-gesture SLK) [22]	Encode strokes as gestures (orthogonal/diagonal/transitional/stylistic) for analysis and regeneration [22]	Symbolic sequence processing; any language [22]	Gesture strings ↔ Kolam drawings [22]	Human-aligned, extensible encoding [22]	Needs comprehensive annotation and coverage [22]
ML classification	KolamNet/KolamNetV2 [9][2]	EfficientNet + attention modules for multi-category Kolam image classification and heritage documentation [2]	PyTorch/TF-style CNN stacks [2]	Images → categories (13 types, etc.) [2]	Documentation, retrieval, metadata [2]	Not directly generative of rules [2]